Logic-Based Deletion Analysis of Java Code Dependencies

A Prototype for Giving Feedback to AI Coding Assistants

Mathis Burger

24 October 2026

Abstract

AI coding assistants change and delete code without knowing reliably what depends on it. This document describes a prototype that encodes code-level dependencies of a Java project as a feature model, translates it to propositional logic and answers deletion questions on this logical encoding: *Can this class or member be deleted? If not, which functionality breaks, and why?* The encoding and five queries (including two MaxSAT queries and an explanation via a minimal unsatisfiable subset) are specified and implemented on top of the SAT solver Sat4j. For the constraints implemented so far, the encoding lies in a fragment that propagation alone can solve; where a SAT solver becomes necessary is one of the open research questions. A source extractor based on JavaParser builds the feature tree and the plain references between code elements. The queries have been checked on a small hand-built example. An end-to-end run from source on the same example works, but answers only some of the queries correctly, because the extractor does not yet produce roots and polymorphism constraints. A run on a real project and an evaluation have not been done.

Nature of this document. This is a write-up of a prototype, not a research paper. Its purpose is to show that the research I propose is implementable and to make its central idea concrete. It does not yet show that the approach works on real projects: no evaluation has been run, and the extractor covers only part of the encoding. The related work in Section 2 is preliminary and mostly based on abstracts.

Use of AI. AI assistants (Claude Code) were used to draft text and code in this project. I checked all code and text personally. The research idea and the design decisions are my own.

1 Motivation and Question

When an AI coding assistant proposes to delete a class, it usually learns whether this was safe only after compiling and running the tests, or not at all. A tool that answers deletion questions *before* the change, and explains its answer, could serve as feedback for the assistant's decision.

The question behind the prototype is:

> Can a propositional encoding of code-level dependencies, solved with a SAT solver where necessary, tell an AI coding assistant, before compilation or testing, whether a proposed deletion breaks the project? Concretely: can class or member C be deleted? If not, which functionality breaks, and why?

The phrase "where necessary" matters: as Section 3.4 shows, the constraints implemented so far do not require a SAT solver. Finding out which rules about code do require one is part of the question.

The approach borrows from two areas. Package managers have long encoded dependencies between packages as propositional formulas and solved them with SAT solvers. Software

product lines describe valid combinations of features as feature models, which also translate to propositional logic. The prototype applies both ideas at the level of classes, fields, constructors and methods.

2 Preliminary Related Work

This section lists the work found so far. It is not a systematic review. For most entries only the abstract has been read.

Feature models and propositional logic. Batory [4] connects feature models, grammars and propositional formulas, so that feature models can be translated to propositional logic and analysed with off-the-shelf SAT solvers. Kuiter et al. [8] show that the choice of CNF transformation affects the correctness and performance of feature-model analyses; the encoding here avoids the question because every constraint is already a single clause. Sundermann et al. [13] evaluate #SAT solvers on industrial feature models, which becomes relevant if one wants to count valid deletion sets.

SAT on code-level references. The closest prior work found is *safe composition* by Thaker et al. [14]: feature modules of a product line reference classes, methods and fields defined in other modules, and SAT is used to check that no product allowed by the feature model contains a reference to an undefined element. Apel et al. [2] and Kästner et al. [7] give type systems that guarantee all variants of a product line are well-typed. How the proposed research relates to this line of work is discussed in Section 7.

SAT-based package dependency solving. Mancinelli et al. [10] encode package installability as a SAT problem, with one Boolean variable per package version, and check whole Debian and Mandriva repositories with it. Abate et al. [1] survey dependency solving in package managers, note that it is NP-complete in non-trivial component models, and observe that SAT-based solving is spreading. Sat4j [5], the solver used here, provides SAT, MaxSAT and pseudo-Boolean solving in Java and has been used for Eclipse's plugin dependency management. Pinckney et al. [11] use Max-SMT to optimise npm dependency solving.

Breaking changes and LLM feedback. BUMP [12] is a benchmark of reproducible breaking dependency updates in Java projects. Roseau [9] detects API breaking changes in Java libraries. CodePlan [3] uses static dependency analysis to plan repository-wide edits made by an LLM; it does not use SAT.

3 Approach

The full specification is in `docs/encoding.md` in the repository. This section summarises it.

3.1 Features

Every feature is one Boolean variable: *true* means the element is present in the code, *false* that it is deleted. The feature tree is

$$\text{project root} \rightarrow \text{package} \rightarrow \text{class} \rightarrow \text{field / constructor / method},$$

with named nested classes as children of their enclosing class. Each overload of a method or constructor is its own feature, identified by its parameter types (e.g. `Report#total(Shape)`). Parameters are not separate features. Anonymous classes, local classes and lambdas are not

features; their references are attributed to the enclosing member. JDK and library types are not modelled.

3.2 Constraints

Every constraint is a single clause; $a \to b$ stands for $\neg a \vee b$.

- **Tree:** the project root is present; every child requires its parent (*child* $\to$ *parent*). All other features are optional and therefore deletion candidates.
- **References:** the innermost feature containing a reference (its *source*) requires the referenced feature: method and constructor calls (bound to the overload the compiler resolves), type usages, inheritance, annotations and field accesses.
- **Polymorphism:** for an interface or abstract class T with implementations $C_1, \ldots, C_n$, an or-group $T \to (C_1 \vee \cdots \vee C_n)$; a call on an abstract method requires the declaration and at least one implementation; a concrete class requires its implementation of every inherited abstract method. Lambdas count as implementations through their enclosing member. Groups without any implementation in the project are skipped.
- **Roots** ("functionality"): every `main` method and every test class is a root and must be present; a root class requires all of its members.
- **Deletion** of a set D: unit clauses $\neg d$ for $d \in D$.

3.3 Queries

Let F be all clauses except the root unit clauses, R the root unit clauses and D the deletion set.

	Question	Query
Q1	Can D be deleted alone?	SAT of $F \wedge R \wedge \neg D$ under the assumption that every feature outside the deleted subtrees stays present
Q2	Can D be deleted together with dependent code?	SAT of $F \wedge R \wedge \neg D$
Q3	Which code must go with D?	MaxSAT: $F \wedge \neg D$ hard, one soft unit clause per remaining feature
Q4	Which functionality breaks?	MaxSAT: $F \wedge \neg D$ hard, one soft unit clause per root
Q5	Why?	Minimal unsatisfiable subset (MUS) of the Q2 formula; each clause keeps the code reference that produced it

3.4 Complexity, and why SAT is used anyway

Every clause of the encoding has at most one negative literal, so the formulas are *dual-Horn*. Their satisfiability can be decided in linear time: negating every variable turns them into Horn formulas, for which Dowling and Gallier [6] give a linear-time algorithm. Moreover, the set of models is closed under union: if a is true in $M_1 \cup M_2$, it is true in some M_i, which then makes one of the positive literals of every clause $\neg a \vee b_1 \vee \cdots \vee b_k$ true in M_i and therefore in the union. Hence a unique maximal model exists, computable by propagating "absent" from D to a fixpoint, and it answers Q2–Q4 at once. This is only a proof sketch and still has to be checked carefully.

For the current encoding, a graph or propagation algorithm therefore gives the same answers as SAT and MaxSAT. Explanations are not an argument for SAT either: in this fragment, the chain of implications that forced a feature to be absent can be read directly from the propagation. Using a solver library is merely convenient here, because it provides minimal unsatisfiable subsets without extra implementation effort. The actual reason to keep a SAT solver is that planned user-defined rules ("exactly one implementation", "A excludes B") contain clauses with more than one negative literal and therefore leave the dual-Horn fragment, so that efficient propagation is no longer guaranteed. This reason is prospective: whether such rules are needed in practice has not been shown. The propagation algorithm will serve as a baseline in the evaluation.

4 Prototype

The prototype is a Maven project in Java 21 with Sat4j 2.3.6 and JavaParser 3.28.2 (including its symbol solver), tested with JUnit 5.

- `FeatureModel` and `Constraint` hold the model in clause form. Each clause keeps a human-readable reason, so that a MUS can be traced back to the code.
- `DeletionQueries` implements Q1–Q5: Q1 and Q2 with Sat4j's default solver (Q1 via assumptions), Q3 and Q4 with Sat4j's weighted MaxSAT decorator, Q5 with Sat4j's `Xplain`.
- `Extractor` builds the model from source in layers. Implemented are layer 1 (the feature tree) and layer 2 (inheritance, type usage, annotations, field access, constructor calls, resolved with the JavaParser symbol solver). Names that cannot be resolved, such as library annotations, are reported as warnings.

Not implemented are layer 3 (method calls and implicit `super()` calls), layer 4 (polymorphism) and layer 5 (roots); see Section 8. The pipeline nevertheless runs end to end on the partial model; Section 6 shows what it gets right and wrong.

5 Worked Example

Example A is a small synthetic program with an interface and two implementations:

```
interface Shape { double area(); }
class Circle implements Shape { public double area() { return 1; } }
class Square implements Shape { public double area() { return 1; } }
class Report { static double total(Shape s) { return s.area(); } }
class ReportTest { @Test void total() { Report.total(new Circle()); } }
```

Its model has 12 features (project root, package `p`, the five classes and their five members) and 24 clauses. The root is `ReportTest`. Three deletions were worked out by hand:

Deletion	Q1	Q2	Q3 cascade	Q4 broken roots
`Square`	yes	yes	`Square`, `Square#area()`	none
`Circle`	no	no	`Circle`, `Circle#area()`, `ReportTest#total()`, `ReportTest`	`ReportTest`
`Square#area()`	no	yes	`Square#area()`, `Square`	none

- `Square` can be deleted because `Circle` still satisfies the or-group of `Shape` and the call `s.area()`. This is the case the or-groups exist for.

- `Circle` cannot be deleted. The explanation (Q5) is the MUS {`ReportTest` is a root; `ReportTest` requires `ReportTest#total()`; `ReportTest#total()` calls `new Circle()`; `Circle` is deleted}.
- `Square#area()` cannot be deleted alone, because `Square` would no longer implement `Shape`; it can be deleted together with `Square`.

6 What Has Been Checked

All numbers in this section come from `mvn test` in the repository (12 tests, all passing on 24 October 2026).

- `ExampleATest` (4 tests): the hand-built model of Example A consists of exactly the 24 documented clauses, and Sat4j's answers to Q1–Q5 for the three deletions above equal the hand-derived ones, including the exact MUS.
- `ExtractorTreeTest` (3 tests): the extracted feature tree of Example A equals the hand-built one; a second fixture covers overloads, generic and varargs parameters, nested classes, enums, interfaces, records and nested packages, and checks that anonymous and local classes, lambdas and initializer blocks do not become features; variable numbering is deterministic.
- `ExtractorReferenceTest` (2 tests): layer 2 produces exactly the four plain-reference clauses of Example A, and 15 hand-derived clauses for a fixture covering inheritance, annotations, generic field types, enum constants, static initializers, explicit `super(...)`, implicit and explicit constructors, casts, class literals, inherited fields, and references from lambdas and anonymous classes.
- `EndToEndTest` (3 tests): the sources of Example A are extracted with layers 1 and 2 only, and the queries run on the resulting model. The test asserts the answers derived by hand for this partial model (`docs/encoding.md`, Section 5), including the ones that are wrong compared with the full model.

These tests show that the encoding, the queries and the first two extractor layers behave as specified on small examples.

End to end with layers 1 and 2 only. The extracted model of Example A contains the feature tree and the four plain references, but no method calls, no or-group, no implementation obligations and no roots. Its answers compare with those of the full model as follows:

Deletion	Q1 alone	Q2 with cascade	Same as full model
`Square`	yes	yes	yes, Q1–Q4
`Circle`	no	yes (full: no)	Q1 only
`Square#area()`	yes (full: no)	yes	Q2 and Q4 only

Deleting `Circle` is still correctly rejected by Q1, because the test calls `new Circle()`. Without roots, however, nothing has to stay present: Q2 accepts the deletion, the cascade (Q3) misses `ReportTest`, no functionality is reported broken (Q4), and there is nothing to explain (Q5). Without the implementation obligation, even Q1 accepts deleting `Square#area()`, although `Square` would then no longer compile. The run therefore shows that the pipeline works from source to answer, and which of the missing layers each answer depends on: roots (layer 5) for Q2, Q4 and Q5, and polymorphism (layer 4) even for Q1.

7 Open Questions and How the Research Would Address Them

The prototype raises several questions that it cannot answer itself. This section states them openly, together with how I would address them in the research. The points in it are arguments and hypotheses, not results.

Why not simply compile and run the tests? Compiling and testing remain the ground truth, and the evaluation would use them as such. The tool is meant to complement them, not to replace them. A compiler answers a question only after the edit has been made, and it reports the errors of that one edit. The queries in this prototype answer questions the compiler does not answer directly: whether a deletion is possible *before* the code is touched, what else would have to be removed with it (Q3), which tests and entry points would be lost (Q4), and why (Q5), as a chain of references rather than a list of error messages. A single model can also be asked about many candidate deletions without rebuilding the project each time. Whether this is faster than incremental compilation in practice, and whether the explanations help an AI assistant, are open questions that the evaluation would have to measure.

Why SAT, if the current encoding is dual-Horn? For the constraints implemented so far, a SAT solver is not necessary (Section 3.4), and the explanations could also be obtained from propagation. I do not see this as a reason to drop the approach, but it changes how the research question should be read: the contribution is a *logical encoding* of code-level dependencies together with deletion queries and explanations, with SAT as a general back end for the cases that need it. A central part of the research would be to find out where the boundary lies, that is, which kinds of rules about code keep the problem in an efficiently solvable fragment and which do not. Candidates that leave the dual-Horn fragment are user-defined rules such as "exactly one implementation of this interface" or "these two components exclude each other". Whether such rules occur often enough in practice to matter is itself an open question. If they do not, a propagation algorithm would be sufficient, and that would be a result worth reporting as well. Until this is known, I favour SAT as the back end, because rules outside the dual-Horn fragment can then be added as further clauses without changing the solver. Such constraints (e.g. $A \wedge B \rightarrow \neg C$) are better handled by SAT than by a custom graph algorithm: propagation alone no longer finds the answer, so the graph algorithm would have to add the search that a SAT solver already provides. How well the SAT and especially the MaxSAT queries scale on real projects has not been measured yet.

What is new compared to safe composition? Thaker et al. [14] already encode code references as propositional constraints between feature modules and check them with SAT. The research would build on this idea rather than claim it. The first intended difference is the granularity: in safe composition a variable stands for a feature module, and a reference only yields an implication between modules, whereas here every class, field, constructor and method is a variable of its own. This is what makes the deletion of a single element expressible at all. Further differences are the input (an ordinary project from which the model is extracted automatically, not a product line with a hand-written feature model), the question (the impact of one concrete deletion, the cascade and the minimal set of broken functionality, rather than type safety of all products) and the consumer (an AI assistant that needs an answer and an explanation). Whether these differences hold up can only be judged after a thorough reading of this line of work, which I plan as a first step.

Where is the AI? The prototype contains no AI. It focuses on the part that has to work first: a reliable answer to a deletion question. The AI-related question, whether an assistant

makes better decisions when it can ask the tool, is a separate experiment. It would only make sense once the tool itself has been evaluated against compilation and test results.

Does it work on real code? Only for one five-class example, and only partly (Section 6). The first milestone of the research would be a complete end-to-end run: completing the extractor, deleting each class of a small open-source subject in turn, and comparing the prediction with the result of compiling and running its tests. False positives and false negatives would be reported as they are, including the ones the current design already predicts (Section 8).

As a first subject I propose java-design-patterns, an open-source collection of the common Java design patterns. Its modules are small and well understood, so answers can be checked by hand, and the patterns they demonstrate recur in real Java code. The modules themselves, however, are demonstrations and are not structured like real projects. To cover that, a large open-source project such as Apache Kafka would be analysed as well; judging whether an answer is correct there requires in-depth knowledge of the codebase, which would have to be built up first. A more focused variant of the research would first analyse which design patterns occur in practice and how relevant they are, and then study how each of them is represented in the encoding.

Two points of the evaluation design need care. First, the ground truth differs per query. For Q1, it is the result of deleting only C and then compiling and running the tests. For Q2 and Q3, it is the result of deleting the predicted cascade instead: the prediction is correct if the project then compiles and only the predicted roots (Q4) are lost. Second, the answers are likely to be imbalanced. In the `strategy` module of the planned subject, for example, the `main` method uses every strategy class, so almost none of them can be deleted. A trivial predictor that always answers "not deletable" would then look accurate. The evaluation would therefore report measures that account for imbalance, such as precision and recall per answer, rather than accuracy alone, and compare against three baselines of increasing strength: "always not deletable"; plain reachability, which treats every reference as required and ignores or-groups, so it cannot see that another implementation takes over; and the propagation algorithm of Section 3.4, which gives the same answers as the encoding in the dual-Horn fragment and therefore isolates the cost and benefit of using a solver. Subjects with more deletable code would also be needed.

Is deletion too narrow? Deletion is a deliberately small starting point because its ground truth is clear. Other changes can partly be expressed through it: changing a method's signature, for example, amounts to deleting one overload and adding another. Whether the approach extends to other kinds of edits is part of the future work.

8 Limitations

- **Partial end-to-end result, no evaluation.** The extractor lacks method calls, polymorphism and roots. From source, only one synthetic example has been run, and several of its answers are wrong for that reason (Section 6). The planned subject (java-design-patterns, Section 7) has not been run.
- **Resolved overload only.** A call requires exactly the overload the compiler picks today. If `bar(int)` is deleted, `bar(5)` still compiles against `bar(long)`, but the model reports a break (false positive).
- **Implicit `super()`** calls are not modelled yet, so deleting a superclass's no-argument constructor can break compilation unnoticed (false negative).
- **Static analysis only.** Reflection, dependency injection, `ServiceLoader`, references from resource files, generated code (e.g. Lombok), runtime-created implementations (mocks,

proxies) and breakage that compiles but fails a test are out of scope. Mocks in particular cause false positives: a test that uses only a Mockito mock of an interface still passes when every implementation is deleted, but the or-group reports it as broken.

- **Approximate signatures.** Overloads are identified by parameter types as written in the source, so type variables are not erased to their bounds and equally named types are not told apart. Identifying each parameter type by its resolved, fully qualified name would fix this.
- **Dual-Horn fragment.** As shown above, the current encoding does not need a SAT solver; the benefit of SAT has to be shown with rules outside this fragment (Section 7).
- **Preliminary literature review.** The related work is incomplete and mostly based on abstracts.

9 Next Steps for the Research

NOTE: Order is not prioritised.

- Complete the extractor (method calls, implicit `super()`, polymorphism, roots) and run the queries end to end.
- Implement the propagation baseline and compare it with SAT.
- Evaluate on open-source projects, starting with java-design-patterns: delete each class in turn, compare the prediction with compiling and running the tests (for Q2 and Q3: after deleting the predicted cascade), and report precision and recall per answer, false positives and negatives, and solver time, compared against trivial baselines and the propagation algorithm.
- Add user-defined rules in UVL constraint syntax (e.g. exactly one implementation), which leave the dual-Horn fragment, and determine the complexity of the resulting problem.
- Render explanations as text for an AI assistant and study whether the assistant decides better with the tool.
- Extend from single projects to services in several languages. Many dependencies between services (e.g. HTTP calls built from configuration values) cannot be proven by static analysis. An AI could infer them, and the encoding would need to include them together with their uncertainty, so that an answer shows which parts rest on proven dependencies and which on inferred ones.
- Carry out a proper literature review, starting with reading the safe-composition and variability-aware type-checking work in full.

References

[1] Pietro Abate, Roberto Di Cosmo, Georgios Gousios, and Stefano Zacchiroli. Dependency solving is still hard, but we are getting better at it. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 547–551, 2020. doi:10.1109/SANER48275.2020.9054837.

[2] Sven Apel, Christian Kästner, Armin Größlinger, and Christian Lengauer. Type safety for feature-oriented product lines. *Automated Software Engineering*, 17(3):251–300, 2010. doi:10.1007/s10515-010-0066-8.

[3] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. CodePlan: Repository-level coding using LLMs and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE):675–698, 2024. doi:10.1145/3643757.

[4] Don Batory. Feature models, grammars, and propositional formulas. In *Software Product Lines (SPLC 2005)*, volume 3714 of *Lecture Notes in Computer Science*, pages 7–20. Springer, 2005. doi:10.1007/11554844_3.

[5] Daniel Le Berre and Anne Parrain. The Sat4j library, release 2.2. *Journal on Satisfiability, Boolean Modeling and Computation*, 7(2–3):59–64, 2010. doi:10.3233/SAT190075.

[6] William F. Dowling and Jean H. Gallier. Linear-time algorithms for testing the satisfiability of propositional Horn formulae. *The Journal of Logic Programming*, 1(3):267–284, 1984. doi:10.1016/0743-1066(84)90014-1.

[7] Christian Kästner, Sven Apel, Thomas Thüm, and Gunter Saake. Type checking annotation-based product lines. *ACM Transactions on Software Engineering and Methodology*, 21(3):1–39, 2012. doi:10.1145/2211616.2211617.

[8] Elias Kuiter, Sebastian Krieter, Chico Sundermann, Thomas Thüm, and Gunter Saake. Tseitin or not Tseitin? The impact of CNF transformations on feature-model analyses. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–13, 2022. doi:10.1145/3551349.3556938.

[9] Corentin Latappy, Thomas Degueule, Jean-Rémy Falleri, Romain Robbes, and Lina Ochoa. Roseau: Fast, accurate, source-based API breaking change analysis in Java. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 517–528, 2025. doi:10.1109/ICSME64153.2025.00053.

[10] Fabio Mancinelli, Jaap Boender, Roberto Di Cosmo, Jérôme Vouillon, Berke Durak, Xavier Leroy, and Ralf Treinen. Managing the complexity of large free and open source package-based software distributions. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*, pages 199–208, 2006. doi:10.1109/ASE.2006.49.

[11] Donald Pinckney, Federico Cassano, Arjun Guha, Jonathan Bell, Massimiliano Culpo, and Todd Gamblin. Flexible and optimal dependency management via Max-SMT. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1418–1429, 2023. doi:10.1109/ICSE48619.2023.00124.

[12] Frank Reyes, Yogya Gamage, Gabriel Skoglund, Benoit Baudry, and Martin Monperrus. BUMP: A benchmark of reproducible breaking dependency updates. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 159–170, 2024. doi:10.1109/SANER60148.2024.00024.

[13] Chico Sundermann, Tobias Heß, Michael Nieke, Paul Maximilian Bittner, Jeffrey M. Young, Thomas Thüm, and Ina Schaefer. Evaluating state-of-the-art #SAT solvers on industrial configuration spaces. *Empirical Software Engineering*, 28(2), 2023. doi:10.1007/s10664-022-10265-9.

[14] Sahil Thaker, Don Batory, David Kitchin, and William Cook. Safe composition of product lines. In *Proceedings of the 6th International Conference on Generative Programming and Component Engineering (GPCE)*, pages 95–104, 2007. doi:10.1145/1289971.1289989.